

Claude Code Agent Architecture – Multi-Agent Patterns

Date: 2026-02-24

Context: Meridian AI Capital chart analysis pipeline

What is an “Agent” in Claude Code?

An agent is a **separate Claude Code subprocess** with its own conversation context. Think of it as opening another terminal tab with Claude Code, giving it a specific task, and letting it work autonomously.

Main Claude Code Process (you're talking to this one)

- └─ Task Agent A ── separate conversation, separate context window
- └─ Task Agent B ── separate conversation, separate context window
- └─ Task Agent C ── separate conversation, separate context window
- └─ Task Agent D ── separate conversation, separate context window
- └─ Task Agent E ── separate conversation, separate context window

Each agent: – Has its own context window (up to 200K tokens) – Makes its own API calls to Anthropic – Can read/write files independently – Can run bash commands independently – Reports results back to the parent when done – Shares the same rate limit pool with all other agents

Agent Types Available

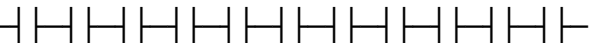
Type	Tools Available	Best For
general-purpose	All tools (Read, Write, Edit, Bash, Glob, Grep, WebSearch, etc.)	Complex multi-step tasks, code generation, analysis
Explore	Read-only tools (Read, Glob, Grep, WebFetch)	Codebase exploration, research, no modifications
Bash	Only Bash command execution	Git operations, running scripts, terminal tasks
Plan	Read-only tools	Architecture planning, design decisions

Concurrency Model

Timeline 

Main: [Launch A,B,C,D,E] — [wait] — [collect results] — [continue]

Agent A:  (34 tickers)
Agent B:  (34 tickers)
Agent C:  (34 tickers)
Agent D:  (34 tickers)
Agent E:  (34 tickers)

API calls:  (interleaved)

All agents run in parallel. Each makes independent API calls. The Anthropic servers handle them concurrently. Rate limits are shared.

Throughput Discovery (from our testing)

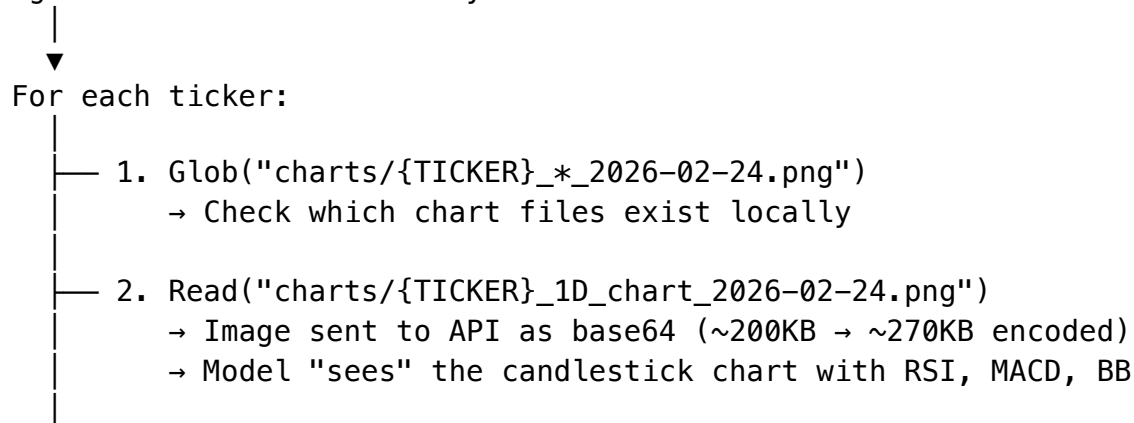
Concurrent Agents	Result
1	Works fine, slow
3	Smooth, no rate limits
5	Smooth, occasional brief pauses
10	Rate limits hit on some agents
15	Multiple agents hit rate limits, some fail

Sweet spot: 5 concurrent agents for Pro Max subscription.

How Chart Analysis Agents Work

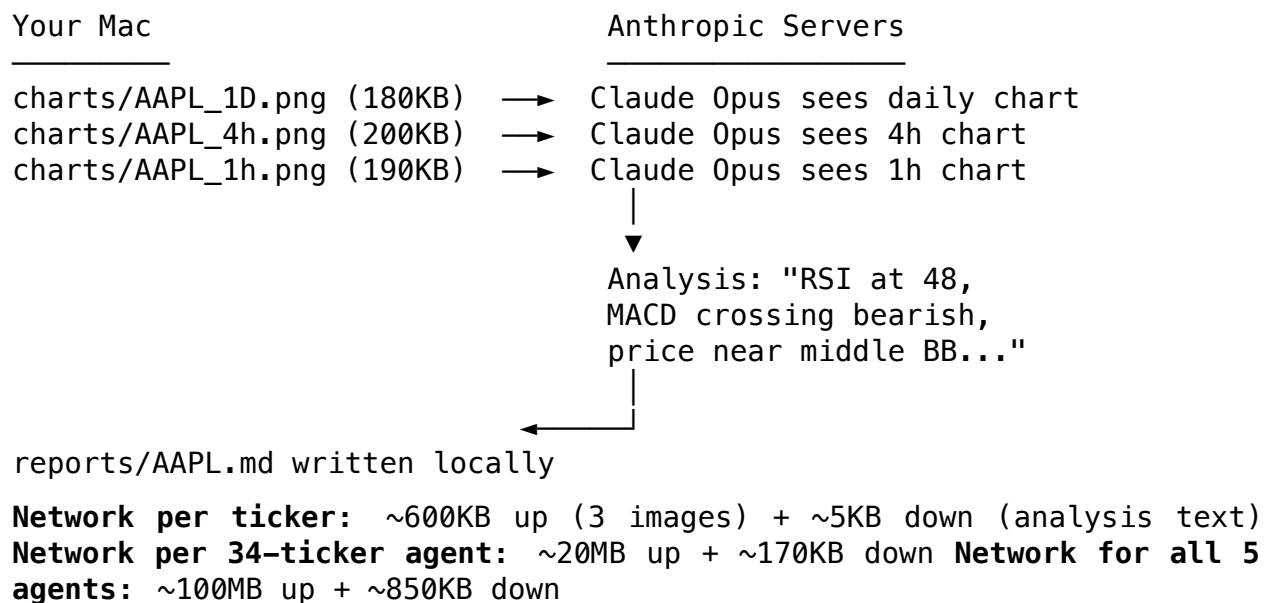
Per-Agent Workflow

Agent receives task: "Analyze 34 tickers"



- 3. Read("charts/{TICKER}_4h_chart_2026-02-24.png")
→ Same – model sees 4-hour chart with VWAP
- 4. Read("charts/{TICKER}_1h_chart_2026-02-24.png")
→ Same – model sees 1-hour chart with VWAP
- 5. Model analyzes all 3 images in context:
 - Identifies trend direction, RSI levels, MACD crossovers
 - Reads Bollinger Band position, VWAP relationship
 - Scores each timeframe -5 to +5
 - Computes weighted composite
- 6. Write("reports/{TICKER}.md", analysis)
→ Report written to local disk

Data Flow for One Ticker



Agent Communication Pattern

Agents are **fire-and-forget with result collection**. There is no inter-agent communication.

Main Process

- spawn(Agent A, task="analyze tickers 1-34") → runs independently
- spawn(Agent B, task="analyze tickers 35-68") → runs independently
- spawn(Agent C, task="analyze tickers 69-102") → runs independently

```
|— spawn(Agent D, task="analyze tickers 103-136") → runs independently
|— spawn(Agent E, task="analyze tickers 137-170") → runs independently
```

... all agents work in parallel ...

```
|— Agent A completes → "wrote 28 reports"
|— Agent B completes → "wrote 31 reports"
|— Agent C completes → "wrote 25 reports"
|— Agent D completes → "wrote 30 reports"
|— Agent E completes → "wrote 27 reports"
```

Main: "Total: 141 new reports written"

No shared state between agents – each works on non-overlapping ticker sets. They write to the same reports/ directory but never to the same file.

Three Pipeline Options (for Meridian AI Capital)

Option 1: Interactive Task Agents (Current)

Human → Claude Code CLI → spawn 5 Task agents → each reads charts → writes

- **Pros:** Zero API cost (Pro Max flat rate), interactive, can inspect as it runs
- **Cons:** Manual trigger, rate-limited to ~5 agents, need active terminal session
- **Best for:** Daily manual analysis, ad-hoc deep dives

Option 2: API Pipeline (Existing)

Python script → Anthropic API → Claude Opus analyzes charts → writes reports

- **Pros:** Fully automated, can run as cron job, model flexibility (Opus/Sonnet/Haiku)
- **Cons:** ~\$0.15-0.20 per ticker (\$150-200/day for 1000 tickers)
- **Best for:** Production, unattended runs, cost-sensitive (use Sonnet)
- **Script:** scripts/run_analysis_on_charts.py

Option 3: Claude Code SDK (Planned)

Python script → `claude_code_sdk.query()` → spawns Claude Code sessions → reads

- **Pros:** Automated + Pro Max flat rate, programmatic control, 3 parallel agents
- **Cons:** Depends on SDK availability, still rate-limited
- **Best for:** Daily automated runs at zero marginal cost
- **Script:** scripts/analyze_charts_sdk.py (to be built)

```

# Option 3 pseudocode
import asyncio
from claude_code_sdk import query

async def analyze_ticker(ticker: str):
    prompt = f"Read charts for {ticker} and write report to reports/{ticker}.md"
    result = await query(prompt=prompt, tools=["Read", "Write", "Glob"])
    return result

async def main():
    semaphore = asyncio.Semaphore(3) # max 3 parallel

    async def bounded(ticker):
        async with semaphore:
            return await analyze_ticker(ticker)

    tickers = load_remaining_tickers()
    await asyncio.gather(*[bounded(t) for t in tickers])

```

Daily Workflow (Complete)

```

6:00 AM  — Fetch charts —————
           python scripts/fetch_charts_bulk.py
           → 1,000 tickers × 3 timeframes = 3,000 API calls
           → ~90 min, uses CHART-IMG ULTRA plan
           → Output: charts/{TICKER}_{TF}_chart_{DATE}.png

8:00 AM  — Analyze charts —————
           Option 1: Launch 5 Task agents interactively
           Option 2: python scripts/run_analysis_on_charts.py
           Option 3: python scripts/analyze_charts_sdk.py
           → Each ticker: read 3 PNGs → score → write report
           → Output: reports/{TICKER}.md

9:30 AM  — Build master summary —————
           python scripts/build_master_summary.py --date today
           → Parses all reports, ranks all tickers
           → Output: 00-MASTER-SUMMARY.md

9:35 AM  — Cross-date comparison —————
           python scripts/compare_ticker_reports.py \
             --from yesterday --to today
           → Shows upgrades, downgrades, signal changes
           → Output: comparison-{from}-vs-{to}.md

9:40 AM  — Review & trade —————

```

Read: 00-MASTER-SUMMARY.md (top buys/sells)
Read: comparison report (what changed overnight)
Drill down: reports/AAPL.md, reports/BTCUSDT.md, etc.

File Structure

```
analysis/
  2026-02-24-comprehensive/
    charts/                                # 1,710 PNG files
      AAPL_1D_chart_2026-02-24.png
      AAPL_4h_chart_2026-02-24.png
      AAPL_1h_chart_2026-02-24.png
      ...
    reports/                              # 550+ individual .md files
      _INDEX.md                          # Master index
      AAPL.md
      MSFT.md
      BTCUSDT.md
      ...
    analysis/                            # Batch reports (legacy)
      batch-01-mega-cap-tech.md
      batch-02-us-banks.md
      ...
      batch-15-remaining.md
    00-MASTER-SUMMARY.md                 # Aggregated scoreboard
    comparison-*-vs-*.md                 # Cross-date diffs

scripts/
  fetch_charts_bulk.py                   # Step 1: Download charts
  run_analysis_on_charts.py              # Step 2a: API analysis
  extract_ticker_reports.py              # Step 2b: Extract from batches
  build_master_summary.py                # Step 3: Aggregate
  compare_ticker_reports.py              # Step 4: Cross-date diff
  analyze_charts_sdk.py                  # Step 2c: SDK pipeline (planned)
```

Key Metrics (2026-02-24 Session)

Metric	Value
Total tickers in universe	1,000
Charts fetched	1,710 (572 tickers × 3 TF)
Tickers analyzed	391 (from batches) + 170 (agents in progress)
Batch reports	15 files, ~600KB total
Individual reports	391 + growing

Metric	Value
CHART-IMG API calls used	~2,700 / 3,000 daily limit
Analysis cost (Pro Max)	\$0 incremental
Analysis cost (API equivalent)	~\$80-110 saved
Agents run concurrently	5 (sweet spot for Pro Max)
Time: chart fetch	~90 min
Time: analysis (5 agents)	~15-20 min
Time: summary + reports	~2 min